



chapter 6

■ ■ ■ ■ ■ ■ ■ ■ ■

指针

1

指针的基本概念

2

指针运算

3

指针与数组

4

指针数组与多级指针

5

指针与函数

6

综合应用

6.1.1

指针的基本概念

内存的地址

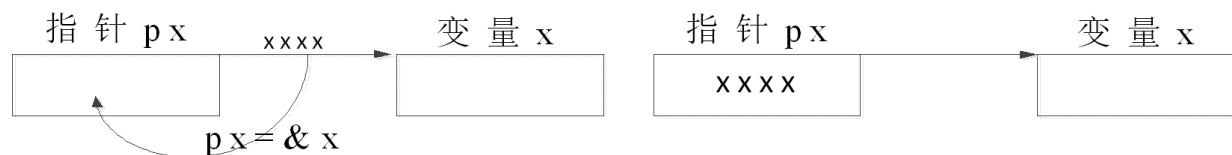
指针

指针是一种数据类型

指针变量

指针常量

例: `int *px, x;`
`px = &x;`



变量的地址装入指针

指针指向变量

指针的目标变量 `x` 是 `px` 的目标变量

变量的访问方式 直接访问方式 `x = 10;`

间接访问方式 `*px = 10;`

指针赋予了C程序**直接访问和修改**内存中数据的能力

0xFFFF FFFF	
0xFFFF FFFE	
0xFFFF FFDD	
0xFFFF FFDC	
0xFFFF FFDB	
0xFFFF FFDA	
0xFFFF FFD9	
0xFFFF FFD8	
...	
0x0000 0007	
0x0000 0006	
0x0000 0005	
0x0000 0004	
0x0000 0003	
0x0000 0002	
0x0000 0001	
0x0000 0000	

指针变量的定义

<存储类型> <数据类型> *指针名;

存储类型是指针变量本身的存储类型

- 也分为auto型，register型、static型和extern型。
- 指针的存储类型和指针声明的位置决定了指针的寿命和可见性。即指针变量也分为内部的和外部的，全局的和局部的。

数据类型说明指针所指向目标的数据类型

例如：

```
int *px;
char *name;
static int *pa;
int *px, *py, *pz;
char cc, *name;
```

具有相同存储类型和数据类型的指针可在一行中说明，也可和普通变量在一起说明

6.1.2

指针变量的定义与初始化

void指针: **<存储类型>** **void *指针名;**

例:

```
int x,*px=&x;  
void *pp;  
pp = px;
```

可以将已定向的各种类型指针直接赋给void型指针

```
int *px;  
void *pp = malloc(100);  
px = (int *)pp;
```

必须采用强制类型转换

指针变量的初始化

<存储类型> **<数据类型>** ***指针名[=初始地址值];**

例如:

```
char cc;  
char *pc = &cc;
```

不要将一个整型量赋给一个指针变量

```
int n;  
int *p=&n;  
int *q=p;  
  
int *pointer=1000;
```

```
#include <stdio.h>  
void main()  
{  
    int a;  
    int *pa = &a; //指针pa指向a所在内存地址  
    a = 10;  
    printf("a:%d\n",a);  
    printf("*pa:%d\n",*pa);  
    printf("&a:0x%lx\n",&a);  
    printf("pa:0x%lx\n",pa);  
    printf("&pa: 0x%lx\n",&pa);  
}
```

运行结果:
a:10
*pa:10
&a:0x1000fff4
pa:0x1000fff4
&pa:0x1000fff8

取地址运算符 &

& 操作对象

操作对象必须是左值表达式（即变量或有名存储区）

单目&与操作对象组成地址表达式，运算结果为操作对象变量的地址，结果类型为操作对象类型的指针

例如

```
int x;  
char y;  
double z;  
int a [4];  
register int k;
```

```
&x  
&y  
&z  
&a[2]  
&a  
&k
```



```
int *  
char *  
double *  
int *  
错误  
错误
```

数组名、常量和寄存器变量均不能做单目&的操作对象

取内容运算符 *

* 操作对象

操作对象必须是地址表达式，即指针（地址常量或指针变量）

运算结果为指针所指的對象，即变量本身（间接访问表达式是左值表达式），结果类型为指针所指对象的类型。

例如 `char c, *pc = &c;`
`*(&c) = 'a';`
`*pc = 'a';`
`c = 'a';`

注意区分*的不同含义：

`char *pc;`

`*pc = 10;`

`a = a*10;`

抽象指针说明符

间访运算符（单目*）

乘运算符

单目*和 &的运算关系

*(& 左值表
&(*地址表

```
1 #include <stdio.h>
2 int main()
3 {
4     int a=1;
5     int *p=&a;
6     *(&a)=2;
7     printf("%d,%d\n", a, *p);
8     printf("%p, %p", &a, &(*p));
9 }
10
```

```
D:\test-c\thefirstproject\bin\Debug\thefirstproject...
2, 2
0000000000061FE14, 0000000000061FE14
Process returned 0 (0x0)   execution time : 0.312 s
Press any key to continue.
```

指针的正确用法

- (1) 必须按被引用变量的类型说明指针变量;
- (2) 必须用被引用变量的地址给指针变量赋值 (或用指针变量初始化方式), **使指针指向确定的目标对象**, 然后才能使用指针来引用变量。

例: `int *p;`
`*p = 5; ?`



```
1  #include <stdio.h>
2  int main()
3  {
4      int *p1, *p2;
5      int a=1, b=2;
6      p1=&a;
7      printf("*p1=%d\n", *p1);
8      *p2=b;
9      printf("*p2=%d\n", *p2);
10     return 0;
11 }
```


NULL指针

ANSI C++标准定义NULL指针，它作为指针变量的一个特殊状态，表示不指向任何东西

使一个指针变量为NULL，可以给它赋一个零值

测试一个指针变量是否为NULL，可以将它和零进行比较

对一个NULL指针进行解引用操作是非法的

```
1  #include <stdio.h>
2  int main()
3  {
4      printf("%d\n", NULL);
5      if("\0"==NULL)
6          printf("Equal!\n");
7      return 0;
8  }
```

```
0
Equal!
```

```
#include <stdio.h>
int main()
{
    int *p=NULL;
    printf("address=%p\n", p);
    *p=1;
    printf("Here\n");
    printf("%d\n", *p);
    return 0;
}
```

```
D:\test-c\thefirstproject\bin\Debug\thefirstproject.exe
address=0000000000000000
Process returned -1073741819 (0xC0000005)   execution time : 3.776 s
Press any key to continue.
```

野指针



- **野指针**的形成原因有两种：
 - 一种是指针变量定义后未初始化
 - 另一种是指针指向了一个已释放的内存空间
- **将未初始化的指针和指向释放的内存空间的指针赋值为NULL，防止意外操作野指针**

void为无类型，“void
*”就是无类型指针，无
类型指针是一种可以指
向任意类型的指针，也
称为通用指针。



6.1.3

指针的使用——通用指针

通用指针指向的内存可存放任意指针类型的数据，程序无法正确解读该内存中的数据
访问void类型指针指向的数据会提示“不允许使用不完整类型”错误
void类型指针在使用时**需要强制转换**为其他类型的指针进行访问



```
1  #include <stdio.h>
2  int main()
3  {
4      int a = 1;
5      int* p = &a;
6      void* pp=p;
7      printf("整型变量a的地址为%p，值为%d\n",&a, a);
8      printf("int类型的指针变量p的地址为%p，指向地址为%p\n",&p, p);
9      printf("void类型的指针变量pp的地址为%p，指向地址为%p\n", &pp,pp);
10     printf("int指针p地址空间的值为：%d\n", *p);
11     printf("void指针pp地址空间的值为：%d\n",*(int*)pp);    //强制转换
12     return 0;
13 }
```

整型变量a的地址为000000000061FE1C，值为1
int类型的指针变量p的地址为000000000061FE10，指向地址为000000000061FE1C
void类型的指针变量pp的地址为000000000061FE08，指向地址为000000000061FE1C
int指针p地址空间的值为：1
void指针pp地址空间的值为：1

例6.2 输入a和b两个整数，按先大后小的顺序来输出a和b

```
#include <stdio.h>
int main()
{
    int *p1,*p2,*p,a,b;
    a = 10;
    printf("please enter tow numbers:");
    scanf("%d%d",&a,&b);
    p1=&a;p2=&b;
    if(a<b)
    {
        p=p1;p1=p2;p2=p;
    }
    printf("a=%d,b=%d\n",a,b);
    printf("max=%d,min=%d\n",*p1,*p2);
    return 0;
}
```

交换两个指针变量的值实现a和b的交换输出

运行结果：

```
please enter two numbers:
5 9
a=5,b=9
max=9,min=5
```

指针的运算种类

算术运算

关系运算

赋值运算

指针可以进行的算术运算：与整数加减

前提：指针
是指向一片
存储单元
(如数组)

$p1+n$

$p1-n$

$p1++$

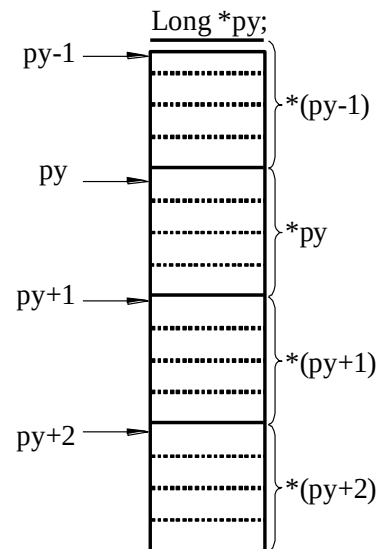
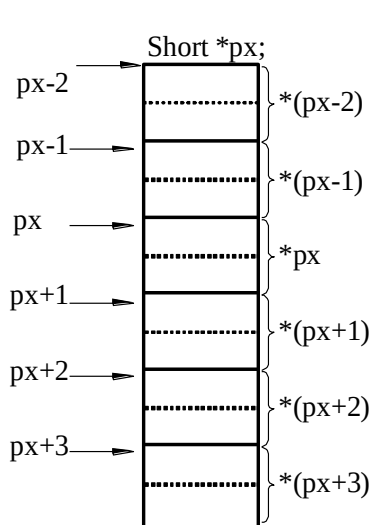
$++p1$

$p1--$

$--p1$

$p1-p2$

指针当前指向位置的后方或前方第 n 个数据的位置，位置的地址值是： $(p1) \pm n \times \text{sizeof}(\text{数据类型})$ (字节)



前

后

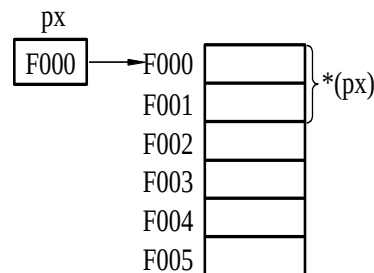
指针可以进行的算术运算：自增和自减

前提：指针是指向一片存储单元（如数组）

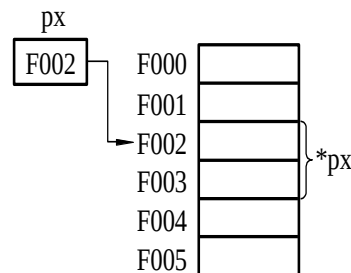
$p1+n$
 $p1-n$
 $p1++$
 $++p1$
 $p1--$
 $--p1$
 $p1-p2$

指向下一个/上一个数据的位置，位置的地址值是： $(p1) \pm \text{sizeof}(\text{数据类型})$ （字节）

Short *px;



px++;



$y = *px++;$

$y = *(px++);$

px的当前目标变量的值赋予y后，px加1指向下一个目标

$y = ++(*px);$

px的目标变量的值加一后赋予y


```
1  #include <stdio.h>
2  int main()
3  {
4      int a[2] = {1,2};
5      int *p=&a[0];
6      int b;
7      printf("1: *p=%d, p at %p, a[2] starts from %p\n", *p, p, a);
8      b=*p++;
9      printf("2: p at %p, b=%d, *p=%d\n", p, b, *p);
10     p++;
11     printf("3: p at %p, *p=%d\n", p, *p);
12     ++p;
13     printf("4: p at %p, *p=%d\n", p, *p);
14     return 0;
15 }
```

```
1: *p=1, p at 000000000061FE0C, a[2] starts from 000000000061FE0C
2: p at 000000000061FE10, b=1, *p=2
3: p at 000000000061FE14, *p=1
4: p at 000000000061FE18, *p=6422040
```

指针可以进行的算术运算：指针相减

前提：指针是指向一片存储单元（如数组）

$p1+n$

$p1-n$

$p1++$

$++p1$

$p1--$

$--p1$

$p1-p2$

```
#include <stdio.h>
int main()
{
    int a[100];
    int *p1=a;
    int *p2=&a[20];
    printf("offset=%d", p2-p1);
    return 0;
}
```

D:\test-c\thefirstproject\bin\Debug\offset=20

```
#include <stdio.h>
int main()
{
    int a[100],b;
    int *p1=a, *p2=&b;
    printf("offset=%d", p2-p1);
    return 0;
}
```

D:\test-c\thefirstproject\bin\Debug\offset=-1

两指针指向的地址位置之间的数据个数： $((p1)-(p2))/\text{sizeof}(\text{数据类型})$

$p1$ 和 $p2$ 指向同一组数据类型一致的数据

```
int a[100],n;
int *p1=a;
int *p2=&a[20];
n = p2-p1;
```

```
int a[100],b,n;
int *p1=a, *p2=&b;
n = p2-p1;
```

```
1  #include <stdio.h>
2  int main()
3  {
4      int a[100], b1, b2; double c; long d;
5      int *pa=a, *pb1=&b1, *pb2=&b2;
6      double *pc=&c;
7      long *pd=&d;
8      printf("pb1-pa=%d\n", pb1-pa);
9      printf("pb2-pa=%d\n", pb2-pa);
10     printf("pb2-pb1=%d\n", pb2-pb1);
11     // printf("pc-pb2=%d\n", pc-pb2); //error: invalid operands to binary
12     // printf("pd-pa=%d\n", pd-pa); //error: invalid operands to binary -
13     return 0;
14 }
```

```
pb1-pa=-1
pb2-pa=-2
pb2-pb1=-1
```

例6.2 通过指针变量输出数组a的5个元素

```
#include <stdio.h>
int main()
{
    int*p,i,a[5];
    p=a;
    printf("please input 5
numbers:\n");
    for (i=0;i<5;i++)
        scanf("%d",p++);
    printf ("\n");
    printf("the input array is:\n");
    for(p=a,i=0;i<5;i++)
        printf("%d ",*p++);
    return 0;
}
```

通过p获得数组
a各元素的地址

通过p访问数组
a各元素的值

运行结果：

please input 5 numbers:
12 34 56 78 90
the input array is:
12 34 56 78 90

两指针之间的关系运算表示它们指向的地址位置之间的关系。
指向后方的指针大于指向前方的指针[存储器的编号从小到大]

$p1 < p2$

成立即p1指向位置在p2指向位置的前面

$p1 > p2$

成立即p1指向位置在p2指向位置的后面

$p1 == p2$

成立即p1和p2指向同一位置

```
1  #include <stdio.h>
2  int main()
3  {
4      int a[100], b1, b2; double c; long d;
5      int *pa=a, *pb1=&b1, *pb2=&b2;
6      double *pc=&c;
7      long *pd=&d;
8      printf("pa>pb1?=%d\n", pa>pb1);
9      printf("pb2>pb1?=%d\n", pb2>pb1);
10     printf("pc>pb2?=%d\n", pc>pb2);
11     printf("pd>pc?=%d\n", pd>pc);
12     return 0;
13 }
```

```
pa>pb1?=1
pb2>pb1?=0
pc>pb2?=0
pd>pc?=0
```

向指针变量赋值时，赋的值必须是地址常量或变量，不能是普通整数

变量的地址赋予一个指向相同数据类型的指针	<pre>char c, *pc; pc=&c;</pre> 地址常量
一个指针的值赋予指向相同数据类型的另一个指针	<pre>int *p, *q; p=q;</pre> 指针变量
数组的地址赋予指向相同数据类型的指针	<pre>char name[20], *pname; pname=name;</pre>
动态内存分配	<pre>int *p, n=20; p=(int *)malloc(n*sizeof(int)); if(p!=NULL) { ... }</pre>

动态内存分配: 在程序运行期间动态地分配存储空间,分配的内存空间放在数据区的堆 (Heap) 中

指定所分配内存空间的大小

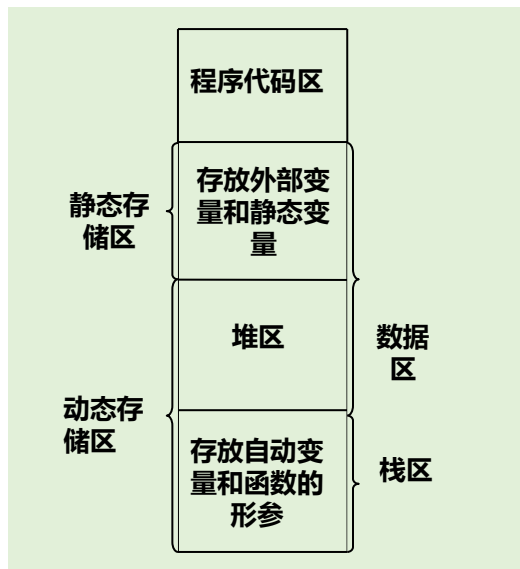
动态内存分配函数: `void * malloc(unsigned size);`

函数运行成功, 则返回值是大小为size的内存空间首地址, 类型是void型指针, 否则, 返回空指针。采用指针赋值操作把它赋值给其它非void指针时, 必须采用类型强制转换。

malloc函数调用一般形式

```
if((指针名 = (类型 *)malloc(空间大小)) == NULL)
{
    出错处理操作
}
```

原型函数在stdlib.h中, 在使用它们的程序开头处, 必须写有:
`#include <stdlib.h>`



动态内存释放函数: `void free(void * ptr);`

接受malloc()函数在堆区中所分配的内存空间首地址

free()是malloc()的配对物，即在整个源程序内，它们是成对出现的

例6.4 使用malloc和free函数的示例程序

```
#include <stdio.h>
#include <malloc.h>
#include <stdlib.h>
int main()
{
    int i = 0, *a, N;
    printf("Input array length:");
    scanf("%d",&N);
    a = (int*)malloc(N*sizeof(int));
    for(i = 0;i<N;i++)
    {
        a[i] = i+1;
        printf("%-5d", a[i]);
        if((i+1)%10 == 0)
            printf("\n");
    }
    free(a);
    printf("\n");
    return 0;
}
```

运行结果:

Input array length:15

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15					